

Proyecto - Algoritmos de Optimización

July 25, 2026

1 Algoritmos de Optimización - Seminario

Autores:

Gonzalo Domínguez Sandoya
Adrián García Sánchez

Repositorio:

<https://github.com/AgSanches/AlgoritmosOptimizaci-nTrabajo>

2 Problema 2. Organizar los horarios de partidos de La Liga

Desde **LaLiga** de fútbol profesional se pretende organizar los horarios de los partidos de cada jornada. Se conocen algunos datos que deben permitir diseñar un algoritmo que realice la asignación de los partidos a los distintos horarios de forma que **maximice la audiencia**.

2.1 Horarios disponibles

Los horarios disponibles son los siguientes:

Día	Horarios
Viernes	20:00
Sábado	12:00, 16:00, 18:00, 20:00
Domingo	12:00, 16:00, 18:00, 20:00
Lunes	20:00

2.2 Categorías y audiencia base

Los equipos se clasifican en tres categorías según su número de seguidores (directamente relacionado con la audiencia):

- **Categoría A:** 3 equipos
- **Categoría B:** 11 equipos
- **Categoría C:** 6 equipos

La siguiente tabla muestra la audiencia estimada (en millones de espectadores) para cada enfrentamiento si el partido se disputa el **sábado a las 20:00**, considerado el mejor horario posible.

	Categoría A	Categoría B	Categoría C
Categoría A	2.00 M	1.30 M	1.00 M
Categoría B		0.90 M	0.75 M
Categoría C			0.47 M

2.3 Ponderación por horario

Si el partido no se disputa el **sábado a las 20:00**, la audiencia se multiplica por el coeficiente correspondiente.

Horario	Viernes	Sábado	Domingo	Lunes
12:00	—	0.55	0.45	—
16:00	—	0.70	0.75	—
18:00	—	0.80	0.85	—
20:00	0.40	1.00	1.00	0.40

Restricción obligatoria: debe programarse **exactamente un partido el viernes** y **exactamente un partido el lunes**.

2.4 Penalización por coincidencias

Es posible programar varios partidos en el mismo horario. En ese caso, la audiencia de **cada partido** se reduce según el número de partidos coincidentes.

Coincidencias	Penalización
0	0 %
1	25 %
2	45 %
3	60 %
4	70 %
5	75 %
6	78 %
7	80 %
8	80 %

(*) La respuesta es obligatoria.

¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones?

Posibilidades teniendo en cuenta todas las restricciones

Para resolver esta cuestión, primero se analizan los elementos de los que se dispone según el enunciado del problema: 1. Hay 10 partidos de fútbol que se deben programar en una jornada. 2. Existen 10 franjas horarias disponibles a lo largo del fin de semana (1 el viernes, 4 el sábado, 4 el domingo y 1 el lunes).

Dado que el enunciado permite que haya solapamientos (es decir, varios partidos pueden coincidir en la misma franja horaria sin invalidar la solución, aunque se aplique una penalización), se trata de un problema matemático de variaciones con repetición.

Para cada uno de los 10 partidos, se puede elegir cualquiera de los 10 horarios disponibles de forma independiente. Por tanto, el cálculo se formula de la siguiente manera:

$$V = m^n = 10^{10}$$

donde V representa el número de variaciones con repetición.

Total de posibilidades sin restricciones:

$$10^{10} = 10\,000\,000\,000$$

es decir, **10 mil millones de combinaciones**.

Posibilidades teniendo en cuenta todas las restricciones

El enunciado indica que existen únicamente dos restricciones obligatorias que invalidarían una solución si no se cumplen: 1. Se debe asignar, al menos, un partido al viernes a las 20h. 2. Se debe asignar, al menos, un partido al lunes a las 20h.

Para calcular matemáticamente las combinaciones válidas, es mucho más sencillo y eficiente utilizar el principio de inclusión-exclusión (calcular los casos totales y restar los casos que rompen las restricciones).

Se definen los siguientes escenarios inválidos: * Conjunto A (sin viernes): combinaciones donde no se juega ningún partido el viernes. Quedan 9 horarios disponibles para los 10 partidos, por lo que hay 9^{10} combinaciones inválidas. * Conjunto B (sin lunes): combinaciones donde no se juega ningún partido el lunes. Quedan 9 horarios disponibles para los 10 partidos, por lo que hay 9^{10} combinaciones inválidas. * Intersección A y B (ni viernes ni lunes): al restar los conjuntos A y B, se está restando dos veces el escenario donde no hay partidos ni el viernes ni el lunes. Se debe sumar esta intersección una vez para equilibrar matemáticamente la ecuación. Si se quitan ambos días, quedan 8 horarios para 10 partidos, resultando en 8^{10} combinaciones.

La fórmula de combinaciones válidas se expresa así:

$$\begin{aligned}\text{Total válidas} &= \text{Total posible} - A - B + (A \cap B) \\ \text{Total válidas} &= 10^{10} - 9^{10} - 9^{10} + 8^{10}\end{aligned}$$

Se realiza el cálculo numérico: $10^{10} = 10.000.000.000$ $9^{10} = 3.486.784.401$ $8^{10} = 1.073.741.824$

$$Total = 10.000.000.000 - 3.486.784.401 - 3.486.784.401 + 1.073.741.824$$

Total de posibilidades teniendo en cuenta las restricciones: 4.100.173.022 combinaciones válidas.

Modelo para el espacio de soluciones (*) ¿Cual es la estructura de datos que mejor se adapta al problema? Argumentalo.(Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)

Respuesta

Para abordar este problema computacionalmente y evaluar las soluciones de forma rápida, la estructura de datos idónea es una combinación de diccionarios y listas, debido a su eficiencia (búsquedas en tiempo constante) y claridad. Se proponen los siguientes elementos.

1. Audiencias base: un diccionario cuyas claves sean tuplas con las categorías de los equipos (ordenadas alfabéticamente) y sus valores sean la audiencia base en millones. Por ejemplo: `audiencias_base = {'A_A': 2.0, 'A_B': 1.3, 'C_C': 0.47, ...}`.
2. Ponderación de horarios: un diccionario donde la clave sea el identificador del horario y el valor sea el coeficiente reductor de audiencia. Por ejemplo: `modificadores_horario = {'Viernes_20': 0.4, 'Sabado_20': 1.0, ...}`.
3. Penalizaciones por coincidencia: una lista simple donde el índice represente el número de coincidencias y el valor sea el porcentaje final de audiencia tras aplicar la reducción. Por ejemplo: `penalizaciones = [1.0, 0.75, 0.55, 0.40, ...]`.
4. Espacio de soluciones: una lista de longitud 10 (una posición por cada partido a jugar) donde el valor almacenado sea la clave del horario asignado. Por ejemplo: `solucion = ['Viernes_20', 'Sabado_20', ...]`.

Esta estructura resulta muy útil porque separa por completo los datos estáticos de la variable dinámica. Esto permite programar la función de evaluación con un código limpio y evitar cálculos redundantes al acceder a los datos mediante una simple consulta directa.

```
[1]: # Estructura de datos para el problema:
```

```
horarios = [  
    'Viernes_20',  
    'Sabado_12', 'Sabado_16', 'Sabado_18', 'Sabado_20',  
    'Domingo_12', 'Domingo_16', 'Domingo_18', 'Domingo_20',  
    'Lunes_20'  
]  
  
partidos = [  
    {'partido': 'Celta - Real Madrid', 'categorias': 'B_A'},  
    {'partido': 'Valencia - R. Sociedad', 'categorias': 'B_A'},  
    {'partido': 'Mallorca - Eibar', 'categorias': 'C_C'},  
    {'partido': 'Athletic - Barcelona', 'categorias': 'B_A'},  
    {'partido': 'Leganés - Osasuna', 'categorias': 'C_C'},  
    {'partido': 'Villarreal - Granada', 'categorias': 'B_C'},  
    {'partido': 'Alavés - Levante', 'categorias': 'B_B'},  
    {'partido': 'Espanyol - Sevilla', 'categorias': 'B_B'},  
    {'partido': 'Betis - Valladolid', 'categorias': 'B_C'},  
    {'partido': 'Atlético - Getafe', 'categorias': 'B_B'}  
]  
  
audiencias_base = {  
    'A_A': 2.0,  
    'A_B': 1.3, 'B_A': 1.3,  
    'A_C': 1.0, 'C_A': 1.0,  
    'B_B': 0.9,  
    'B_C': 0.75, 'C_B': 0.75,  
    'C_C': 0.47  
}  
  
ponderaciones = {  
    'Viernes_20': 0.4,  
    'Sabado_12': 0.55,  
    'Sabado_16': 0.7,  
    'Sabado_18': 0.8,  
    'Sabado_20': 1.0,  
    'Domingo_12': 0.45,  
    'Domingo_16': 0.75,  
    'Domingo_18': 0.85,  
    'Domingo_20': 1.0,  
    'Lunes_20': 0.4  
}  
  
penalizaciones = {  
    0: 1.0,    # 0% de reducción  
    1: 0.75,   # 25% de reducción
```

```

2: 0.55, # 45% de reducción
3: 0.40, # 60% de reducción
4: 0.30, # 70% de reducción
5: 0.25, # 75% de reducción
6: 0.22, # 78% de reducción
7: 0.20, # 80% de reducción
8: 0.20, # 80% de reducción
9: 0.20 # 80% de reducción
}

```

Según el modelo para el espacio de soluciones (*)¿Cual es la función objetivo?

(*)¿Es un problema de maximización o minimización?

Respuesta

Según el modelo para el espacio de soluciones (*) ¿cuál es la función objetivo?

La función objetivo es la expresión matemática que calcula la audiencia total de una jornada en base a los horarios asignados al conjunto de partidos, aplicando los modificadores pertinentes. Se formula de la siguiente manera:

$$Audiencia\ Total = \sum_{i=1}^{10} (Base_i \times Ponderacion_Horario_i \times Penalizacion_Coincidencia_i)$$

En esta fórmula, las variables que intervienen se definen del siguiente modo: * $Base_i$: es la audiencia base del partido i según las categorías de los equipos que se enfrentan. * $Ponderacion_Horario_i$: es el coeficiente de reducción de audiencia que se aplica por jugar en un día y hora específicos. * $Penalizacion_Coincidencia_i$: es el porcentaje de audiencia restante tras aplicar la penalización correspondiente si existen otros partidos programados exactamente a la misma hora.

(*) ¿Es un problema de maximización o minimización?

Tal y como se establece en la descripción general del caso práctico, el propósito principal es organizar los horarios de los partidos de liga de cada jornada para conseguir la mayor cantidad de espectadores posibles. Por tanto, es un problema de maximización.

Diseño de un algoritmo por fuerza bruta

```
[2]: def evaluar_audiencia(solucion, partidos, audiencias_base, ponderaciones,
    ↪penalizaciones):
    audiencia_total = 0

    # se recorren los 10 partidos programados en el calendario propuesto
    for i in range(len(solucion)):
        horario_asignado = solucion[i]

        # 1. obtener audiencia base según las categorías de los equipos
        base = audiencias_base[partidos[i]['categorias']]

        # 2. obtener modificador multiplicador por el día y hora jugados
        modificador = ponderaciones[horario_asignado]

        # 3. calcular penalización contando cuántos partidos coinciden a esa
    ↪misma hora
        # (se resta 1 para que el partido no se cuente a sí mismo como
    ↪coincidencia)
        coincidencias = solucion.count(horario_asignado) - 1
        penalizacion = penalizaciones[coincidencias]

        # se aplica la fórmula matemática de la función objetivo
        audiencia_partido = base * modificador * penalizacion

        # se acumula al total de la jornada
        audiencia_total += audiencia_partido

    return audiencia_total

def fuerza_bruta_liga(partidos, horarios, audiencias_base, ponderaciones,
    ↪penalizaciones):
    mejor_audiencia = 0
    mejor_solucion = None

    # generación exhaustiva mediante bucles anidados para los 10 partidos
    for p1 in horarios:
        for p2 in horarios:
            for p3 in horarios:
                for p4 in horarios:
                    for p5 in horarios:
                        for p6 in horarios:
                            for p7 in horarios:
                                for p8 in horarios:
                                    for p9 in horarios:
                                        for p10 in horarios:
```

```

# se agrupan los horarios asignados
→ en una lista
solucion = [p1, p2, p3, p4, p5, p6,
→ p7, p8, p9, p10]

# comprobar restricciones
→ obligatorias
if 'Viernes_20' not in solucion or
→ 'Lunes_20' not in solucion:
    continue

# evaluar la función objetivo
audiencia_actual =
→ evaluar_audiencia(solucion, partidos, audiencias_base, ponderaciones,
→ penalizaciones)

# actualizar si es la mejor hasta
→ ahora
if audiencia_actual >
→ mejor_audiencia:
    mejor_audiencia =
→ audiencia_actual
    mejor_solucion = solucion

return mejor_solucion, mejor_audiencia

```

Aunque nos gustaría mostrar la ejecución del algoritmo, debido al gran coste computacional, resulta muy complicado ejecutarlo y mostrar sus resultados porque podría tardar una gran cantidad de tiempo en ejecutarse. Por lo tanto, solo se muestra como referencia para comparar con otros algoritmos en apartados posteriores.

Calcula la complejidad del algoritmo por fuerza bruta

Complejidad Fuerza bruta: Al anidar 10 bucles iterando sobre 10 horarios posibles, el número total de combinaciones generadas se multiplica en cada paso, resultando en un coste de m^n , es decir, 10^{10} combinaciones.

Además, por cada solución que resulta ser válida, el algoritmo se ve obligado a recorrer la lista de 10 partidos para calcular la suma de la audiencia y aplicar las penalizaciones, lo que añade un coste de tiempo lineal $O(n)$ por cada combinación.

La complejidad teórica total del algoritmo se define así: $O(nm^n)$.

Aplicado a los datos concretos de este problema, la complejidad dictamina que se tendrían que realizar decenas de miles de millones de operaciones básicas. Al observar la forma del código se evidencia visualmente por qué el tiempo de cómputo para los bucles anidados se vuelve inviable para grandes volúmenes de datos, justificando así la utilización de un algoritmo heurístico optimizado en su lugar.

(*) Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza. Argumenta porque crees que mejora el algoritmo por fuerza bruta

Respuesta

Mejora del algoritmo Para este algoritmo que mejore al de fuerza bruta, se ha optado por elegir *Greedy* (voraz), la idea consiste en ordenar la jornada dada por número de espectadores base, en función de la categoría de los equipos implicados y por otra parte para cada horario, su coeficiente de audiencia.

Posteriormente se otorga al horario con mayor coeficiente, el partido con el mayor número de espectadores esperados hasta rellenar la jornada al completo.

Análisis de las penalizaciones por coincidencia:

La función objetivo que modela el problema es la siguiente:

$$Audiencia\ Total = \sum_{i=1}^{10} (Base_i \times Ponderacion_Horario_i \times Penalizacion_Coincidencia_i)$$

Aunque formalmente el espacio de soluciones permite el solapamiento de partidos en una misma franja horaria aceptando una penalización, en este problema particular se demuestra analíticamente que la solución óptima global pertenece exclusivamente al conjunto de soluciones sin coincidencias ($Penalizacion_Coincidencia_i = 1$).

Esto se debe a la naturaleza y relación de los parámetros estáticos del enunciado: 1. **Severidad de la penalización:** el castigo por introducir una sola coincidencia es extremadamente estricto, reduciendo la audiencia un 25% de golpe (aplicando un factor corrector de 0.75). 2. **Gradualidad de los horarios:** los coeficientes de los horarios descienden de manera muy progresiva a lo largo del fin de semana (de 1.0 a 0.85, 0.80, 0.75, etc.).

Debido a esta estructura de datos, el beneficio potencial de desplazar un partido a una franja horaria superior nunca llega a compensar la pérdida del 25% de la audiencia combinada que sufren los partidos solapados. Para la distribución de audiencias base de nuestra liga (cuyo rango se acota entre 2.0 y 0.47 millones), se verifica consistentemente que:

$$F_{con\ coincidencias} < F_{sin\ coincidencias}$$

Dado que disponemos de exactamente 10 partidos y 10 franjas horarias asignables, siempre es posible construir una solución biyectiva (1 partido en cada franja) donde cada partido ocupe un horario único y libre de penalizaciones.

Por lo tanto, el diseño de nuestro algoritmo voraz prescinde justificadamente de la exploración de solapamientos horarios. Esto nos permite garantizar la obtención del óptimo global con una complejidad temporal de tan solo $O(n \log n)$ (determinada por el coste de ordenación), reduciendo drásticamente el tiempo de cómputo frente a la fuerza bruta.

```

[3]: def greedy_audiencia_base_por_partido(partidos, audiencias_base):
    audiencia_por_partido = []

    for idx, partido in enumerate(partidos):
        categoria_partido = partido["categorias"]
        audiencia_por_partido.append({"idx": idx, "audiencia_base":
↪audiencias_base[categoria_partido]})

    return sorted(
        audiencia_por_partido,
        key = lambda x: x["audiencia_base"],
        reverse=True
    )

def evaluar_audiencia_greedy(solucion, partidos, audiencias_base, ponderaciones):
    audiencia_total = 0

    for idx, horario in enumerate(solucion):
        audiencia_base = audiencias_base[partidos[idx]["categorias"]]
        audiencia_total += audiencia_base * ponderaciones[horario]

    return audiencia_total

def greedy_liga(partidos, audiencias_base, ponderaciones):
    # Se ordenan los partidos por audiencia base.
    partidos_ordenados_audiencia = greedy_audiencia_base_por_partido(partidos,
↪audiencias_base)

    # Para cada horario, se observa sus coeficientes de audiencia
    horarios_ordenados = [
        horario
        for horario, _ in sorted(
            ponderaciones.items(),
            key=lambda x: x[1],
            reverse=True
        )
    ]
    solucion = [None] * len(partidos_ordenados_audiencia)

    # Se asigna al mejor partido al mejor horario disponible.
    for partido, horario in zip(partidos_ordenados_audiencia, horarios_ordenados):
        solucion[partido["idx"]] = horario

    audiencia = evaluar_audiencia_greedy(
        solucion,
        partidos,
        audiencias_base,

```

```

        ponderaciones
    )

    return solucion, audiencia

solucion, audiencia_jornada = greedy_liga(partidos, audiencias_base,
    ↪ponderaciones)
for i, horario in enumerate(solucion):
    print(f"{partidos[i]['partido']} -> {horario}")

print(f"\nAudiencia total para la jornada: {audiencia_jornada:.2f} Mill.")

```

```

Celta - Real Madrid -> Sabado_20
Valencia - R. Sociedad -> Domingo_20
Mallorca - Eibar -> Viernes_20
Athletic - Barcelona -> Domingo_18
Leganés - Osasuna -> Lunes_20
Villarreal - Granada -> Sabado_12
Alavés - Levante -> Sabado_18
Espanyol - Sevilla -> Domingo_16
Betis - Valladolid -> Domingo_12
Atlético - Getafe -> Sabado_16

```

Audiencia total para la jornada: 6.86 Mill.

(*)Calcula la complejidad del algoritmo

Respuesta

2.5 Complejidad

En este algoritmo se cuenta con algoritmos de ordenación para ordenar los partidos según su audiencia base, es decir que equipos están involucrados y cuanta afición atraen al partido. Por otra parte, se ordenan también los horarios en base a su ponderación. Como bien se conoce, estas operaciones generan una complejidad de: $O(n \log n)$.

También se recorren los partidos de la jornada para su posterior asignación, la creación de la lista vacía para la solución y por lo tanto: $O(n)$.

Se cuentan con varias operaciones: $O(1)$, las cuales se pueden despreciar. Tomando todo esto en cuenta, la complejidad total del algoritmo es:

$$O(n \log n)$$

Esta complejidad supone una mejora muy significativa respecto al algoritmo de fuerza bruta, implementación que explora todas las posibles asignaciones de horarios para los partidos, con una exploración de:

$$10^{10}$$

combinaciones/jornadas. Por ello, el algoritmo voraz reduce en gran medida el tiempo de ejecución además de obtener una solución óptima.

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Datos de entrada

En primer lugar, vamos a generar una clase para definir los “Equipos” con tres atributos: - ID - Nombre - Categoría

Para que a continuación nos resulte más sencillo generar la jornada en cuestión. Y se genera una lista con todos los equipos y sus categorías correspondientes.

Respuesta

```
[4]: from dataclasses import dataclass
import random
import math

@dataclass(frozen=True)
class Equipo:
    id: int
    nombre: str
    categoria: str

equipos = [
    Equipo(1, "Real Madrid", "A"),
    Equipo(2, "Barcelona", "A"),
    Equipo(3, "Real Sociedad", "A"),

    Equipo(4, "Atlético", "B"),
    Equipo(5, "Athletic", "B"),
    Equipo(6, "Valencia", "B"),
    Equipo(7, "Villarreal", "B"),
    Equipo(8, "Alavés", "B"),
    Equipo(9, "Espanyol", "B"),
    Equipo(10, "Betis", "B"),
    Equipo(11, "Celta", "B"),
    Equipo(12, "Levante", "B"),
    Equipo(13, "Getafe", "B"),
    Equipo(14, "Sevilla", "B"),

    Equipo(15, "Mallorca", "C"),
    Equipo(16, "Eibar", "C"),
```

```

Equipo(17, "Leganés", "C"),
Equipo(18, "Osasuna", "C"),
Equipo(19, "Granada", "C"),
Equipo(20, "Valladolid", "C"),

```

```

]
```

2.5.1 Generar la jornada

Para generar jornadas de manera aleatoria implementamos el método `generar_jornada`, el cual a partir de un conjunto de equipos genera una jornada con los equipos de una manera totalmente aleatoria.

Con ello podemos probar el algoritmo con jornadas aleatorias.

```

[6]: def generar_jornada(equipos):
    partidos = []
    copy_equipos = equipos[:]

    random.shuffle(copy_equipos)

    for i in range(0, len(copy_equipos), 2):
        local = copy_equipos[i]
        visitante = copy_equipos[i + 1]

        partidos.append({
            "partido": f"{local.nombre} - {visitante.nombre}",
            "categorias": f"{local.categoria}_{visitante.categoria}"
        })

    return partidos

jornada = generar_jornada(equipos)
solucion, audiencia_jornada = greedy_liga(jornada, audiencias_base,
↳ ponderaciones)

for i, horario in enumerate(solucion):
    print(f"{jornada[i]['partido']} -> {horario}")

print(f"\nAudiencia total para la jornada: {audiencia_jornada:.2f} Mill.")

```

Celta - Leganés -> Sabado_12
Real Sociedad - Alavés -> Sabado_20
Getafe - Athletic -> Sabado_18
Levante - Mallorca -> Domingo_12
Espanyol - Real Madrid -> Domingo_20
Atlético - Villarreal -> Domingo_16
Eibar - Barcelona -> Domingo_18
Valladolid - Osasuna -> Lunes_20
Betis - Sevilla -> Sabado_16
Granada - Valencia -> Viernes_20

Audiencia total para la jornada: 6.71 Mill.

2.5.2 Generar calendario completo

El calendario de la liga de fútbol consta de 38 partidos si contamos con 20 equipos, donde cada equipo juega de local y visitante ante cada equipo. Desde al año 2018 se cuenta con un calendario totalmente asimétrico, es decir los partidos de la primera y segunda vuelta no se juegan en el mismo orden, es decir no hay relación entre ambas vueltas.

Pero para generar un calendario asimétrico se necesita un algoritmo bastante complejo de implementar con algunas restricciones, por lo tanto se propone adoptar el algoritmo simétrico para generar las jornadas de una manera sencilla y evaluar únicamente el algoritmo de optimización que fue implementado anteriormente.

Para implementar el calendario simétrico, se necesita de un algoritmo “circular” conocido como “Round robin” [2], donde cada equipo obtiene un orden y se van emparejando los equipos según las posiciones y en cada jornada se cambia el orden de una manera circular y se vuelven a emparejar los equipos.

Únicamente hacen falta generar las primeras 19 jornadas y las siguientes 19 serán iguales pero invirtiendo el orden de local y visitante.

Posteriormente utilizamos el algoritmo de optimización para asignar horarios, observar la audiencia potencial por jornada y al final se hace el recuento de la audiencia de la liga al completo.

```
[7]: # Round Robin

def generar_jornada_robin(equipos_jornada, invertir_local_visitante = False):
    partidos = []
    partidos_jornada = len(equipos_jornada) // 2

    for i in range(partidos_jornada):
        local = i
        visitante = -(i + 1)

        if invertir_local_visitante:
            local, visitante = visitante, local

        local = equipos_jornada[local]
```

```

        visitante = equipos_jornada[visitante]

        partidos.append({
            "partido": f"{local.nombre} - {visitante.nombre}",
            "categorias": f"{local.categoria}_{visitante.categoria}"
        })

    return partidos

def generar_calendario_robin(equipos_calendario):
    # Calcular cantidad de jornadas ida.
    cantidad_equipos = len(equipos_calendario)
    cantidad_de_jornadas = cantidad_equipos - 1
    calendario = [None] * (cantidad_de_jornadas * 2)

    equipos_round_robin = equipos_calendario[:]
    random.shuffle(equipos_round_robin)

    for i in range(cantidad_de_jornadas):
        calendario[i] = generar_jornada_robin(equipos_round_robin)
        calendario[cantidad_de_jornadas + i] =
        ↪generar_jornada_robin(equipos_round_robin, invertir_local_visitante=True)

        robin_fijo = equipos_round_robin[0:1]
        robin_variable = equipos_round_robin[-1:] + equipos_round_robin[1:-1]
        equipos_round_robin = robin_fijo + robin_variable

    return calendario

# Generamos el calendario
calendario_partido = generar_calendario_robin(equipos)
suma_audiencia_total = 0

for idx_jornada, jornada in enumerate(calendario_partido):
    solucion, audiencia_jornada = greedy_liga(jornada, audiencias_base,
    ↪ponderaciones)
    suma_audiencia_total += audiencia_jornada

    print(f"\n{'=' * 20} Jornada {idx_jornada + 1} {'=' * 20}")
    for i, horario in enumerate(solucion):
        print(f"{jornada[i]['partido']:<35} → {horario}")

    print("-" * 60)
    print(f"Audiencia de la jornada: {audiencia_jornada:.2f} millones de
    ↪espectadores.\n\n")

print("\n" + "=" * 60)

```

```
print(f"Audiencia total del calendario: {suma_audiencia_total:.2f} millones de_
↳espectadores")
print("=" * 60)
```

===== Jornada 1 =====

Espanyol - Atlético	→ Sabado_18
Real Sociedad - Getafe	→ Sabado_20
Sevilla - Betis	→ Domingo_16
Granada - Eibar	→ Lunes_20
Leganés - Villarreal	→ Sabado_12
Levante - Valencia	→ Sabado_16
Valladolid - Celta	→ Domingo_12
Barcelona - Osasuna	→ Domingo_18
Real Madrid - Athletic	→ Domingo_20
Mallorca - Alavés	→ Viernes_20

Audiencia de la jornada: 6.71 millones de espectadores.

===== Jornada 2 =====

Espanyol - Getafe	→ Sabado_18
Atlético - Betis	→ Domingo_16
Real Sociedad - Eibar	→ Domingo_20
Sevilla - Villarreal	→ Sabado_16
Granada - Valencia	→ Sabado_12
Leganés - Celta	→ Domingo_12
Levante - Osasuna	→ Viernes_20
Valladolid - Athletic	→ Lunes_20
Barcelona - Alavés	→ Sabado_20
Real Madrid - Mallorca	→ Domingo_18

Audiencia de la jornada: 6.52 millones de espectadores.

===== Jornada 3 =====

Espanyol - Betis	→ Domingo_18
Getafe - Eibar	→ Sabado_12
Atlético - Villarreal	→ Sabado_18
Real Sociedad - Valencia	→ Domingo_20
Sevilla - Celta	→ Domingo_16
Granada - Osasuna	→ Viernes_20
Leganés - Athletic	→ Domingo_12
Levante - Alavés	→ Sabado_16
Valladolid - Mallorca	→ Lunes_20
Barcelona - Real Madrid	→ Sabado_20

Audiencia de la jornada: 7.22 millones de espectadores.

===== Jornada 4 =====

Espanyol - Eibar	→ Domingo_12
Betis - Villarreal	→ Sabado_18
Getafe - Valencia	→ Domingo_16
Atlético - Celta	→ Sabado_16
Real Sociedad - Osasuna	→ Domingo_20
Sevilla - Athletic	→ Sabado_12
Granada - Alavés	→ Viernes_20
Leganés - Mallorca	→ Lunes_20
Levante - Real Madrid	→ Sabado_20
Valladolid - Barcelona	→ Domingo_18

Audiencia de la jornada: 6.50 millones de espectadores.

===== Jornada 5 =====

Espanyol - Villarreal	→ Sabado_18
Eibar - Valencia	→ Sabado_12
Betis - Celta	→ Domingo_16
Getafe - Osasuna	→ Domingo_12
Atlético - Athletic	→ Sabado_16
Real Sociedad - Alavés	→ Sabado_20
Sevilla - Mallorca	→ Viernes_20
Granada - Real Madrid	→ Domingo_20
Leganés - Barcelona	→ Domingo_18
Levante - Valladolid	→ Lunes_20

Audiencia de la jornada: 6.52 millones de espectadores.

===== Jornada 6 =====

Espanyol - Valencia	→ Domingo_18
Villarreal - Celta	→ Sabado_18
Eibar - Osasuna	→ Viernes_20
Betis - Athletic	→ Domingo_16
Getafe - Alavés	→ Sabado_16
Atlético - Mallorca	→ Sabado_12
Real Sociedad - Real Madrid	→ Sabado_20
Sevilla - Barcelona	→ Domingo_20
Granada - Valladolid	→ Lunes_20
Leganés - Levante	→ Domingo_12

Audiencia de la jornada: 7.22 millones de espectadores.

===== Jornada 7 =====

Espanyol - Celta	→ Sabado_18
Valencia - Osasuna	→ Sabado_12
Villarreal - Athletic	→ Domingo_16
Eibar - Alavés	→ Domingo_12
Betis - Mallorca	→ Viernes_20
Getafe - Real Madrid	→ Sabado_20
Atlético - Barcelona	→ Domingo_20
Real Sociedad - Valladolid	→ Domingo_18
Sevilla - Levante	→ Sabado_16
Granada - Leganés	→ Lunes_20

Audiencia de la jornada: 6.71 millones de espectadores.

===== Jornada 8 =====

Espanyol - Osasuna	→ Sabado_12
Celta - Athletic	→ Sabado_18
Valencia - Alavés	→ Domingo_16
Villarreal - Mallorca	→ Domingo_12
Eibar - Real Madrid	→ Domingo_20
Betis - Barcelona	→ Sabado_20
Getafe - Valladolid	→ Viernes_20
Atlético - Levante	→ Sabado_16
Real Sociedad - Leganés	→ Domingo_18
Sevilla - Granada	→ Lunes_20

Audiencia de la jornada: 6.52 millones de espectadores.

===== Jornada 9 =====

Espanyol - Athletic	→ Sabado_18
Osasuna - Alavés	→ Sabado_16
Celta - Mallorca	→ Sabado_12
Valencia - Real Madrid	→ Sabado_20
Villarreal - Barcelona	→ Domingo_20
Eibar - Valladolid	→ Lunes_20
Betis - Levante	→ Domingo_16
Getafe - Leganés	→ Domingo_12
Atlético - Granada	→ Viernes_20
Real Sociedad - Sevilla	→ Domingo_18

Audiencia de la jornada: 6.86 millones de espectadores.

===== Jornada 10 =====

Espanyol - Alavés	→ Sabado_18
Athletic - Mallorca	→ Sabado_12
Osasuna - Real Madrid	→ Domingo_18
Celta - Barcelona	→ Sabado_20
Valencia - Valladolid	→ Domingo_12
Villarreal - Levante	→ Domingo_16
Eibar - Leganés	→ Lunes_20
Betis - Granada	→ Viernes_20
Getafe - Sevilla	→ Sabado_16
Atlético - Real Sociedad	→ Domingo_20

Audiencia de la jornada: 6.71 millones de espectadores.

===== Jornada 11 =====

Espanyol - Mallorca	→ Sabado_16
Alavés - Real Madrid	→ Sabado_20
Athletic - Barcelona	→ Domingo_20
Osasuna - Valladolid	→ Lunes_20
Celta - Levante	→ Sabado_18
Valencia - Leganés	→ Sabado_12
Villarreal - Granada	→ Domingo_12
Eibar - Sevilla	→ Viernes_20
Betis - Real Sociedad	→ Domingo_18
Getafe - Atlético	→ Domingo_16

Audiencia de la jornada: 6.86 millones de espectadores.

===== Jornada 12 =====

Espanyol - Real Madrid	→ Sabado_20
Mallorca - Barcelona	→ Domingo_18
Alavés - Valladolid	→ Sabado_12
Athletic - Levante	→ Sabado_18
Osasuna - Leganés	→ Lunes_20
Celta - Granada	→ Domingo_12
Valencia - Sevilla	→ Domingo_16
Villarreal - Real Sociedad	→ Domingo_20
Eibar - Atlético	→ Viernes_20
Betis - Getafe	→ Sabado_16

Audiencia de la jornada: 6.71 millones de espectadores.

===== Jornada 13 =====

Espanyol - Barcelona	→ Sabado_20
Real Madrid - Valladolid	→ Domingo_18
Mallorca - Levante	→ Sabado_16
Alavés - Leganés	→ Sabado_12
Athletic - Granada	→ Domingo_12
Osasuna - Sevilla	→ Viernes_20
Celta - Real Sociedad	→ Domingo_20
Valencia - Atlético	→ Sabado_18
Villarreal - Getafe	→ Domingo_16
Eibar - Betis	→ Lunes_20

Audiencia de la jornada: 6.72 millones de espectadores.

===== Jornada 14 =====

Espanyol - Valladolid	→ Sabado_12
Barcelona - Levante	→ Sabado_20
Real Madrid - Leganés	→ Domingo_18
Mallorca - Granada	→ Lunes_20
Alavés - Sevilla	→ Sabado_18
Athletic - Real Sociedad	→ Domingo_20
Osasuna - Atlético	→ Domingo_12
Celta - Getafe	→ Domingo_16
Valencia - Betis	→ Sabado_16
Villarreal - Eibar	→ Viernes_20

Audiencia de la jornada: 6.71 millones de espectadores.

===== Jornada 15 =====

Espanyol - Levante	→ Sabado_18
Valladolid - Leganés	→ Lunes_20
Barcelona - Granada	→ Domingo_20
Real Madrid - Sevilla	→ Sabado_20
Mallorca - Real Sociedad	→ Domingo_18
Alavés - Atlético	→ Domingo_16
Athletic - Getafe	→ Sabado_16
Osasuna - Betis	→ Domingo_12
Celta - Eibar	→ Viernes_20
Valencia - Villarreal	→ Sabado_12

Audiencia de la jornada: 6.50 millones de espectadores.

===== Jornada 16 =====

Espanyol - Leganés	→ Domingo_16
Levante - Granada	→ Sabado_16
Valladolid - Sevilla	→ Sabado_12
Barcelona - Real Sociedad	→ Sabado_20
Real Madrid - Atlético	→ Domingo_20
Mallorca - Getafe	→ Domingo_12
Alavés - Betis	→ Domingo_18
Athletic - Eibar	→ Viernes_20
Osasuna - Villarreal	→ Lunes_20
Celta - Valencia	→ Sabado_18

Audiencia de la jornada: 7.22 millones de espectadores.

===== Jornada 17 =====

Espanyol - Granada	→ Sabado_16
Leganés - Sevilla	→ Sabado_12
Levante - Real Sociedad	→ Sabado_20
Valladolid - Atlético	→ Domingo_12
Barcelona - Getafe	→ Domingo_20
Real Madrid - Betis	→ Domingo_18
Mallorca - Eibar	→ Lunes_20
Alavés - Villarreal	→ Sabado_18
Athletic - Valencia	→ Domingo_16
Osasuna - Celta	→ Viernes_20

Audiencia de la jornada: 6.86 millones de espectadores.

===== Jornada 18 =====

Espanyol - Sevilla	→ Sabado_18
Granada - Real Sociedad	→ Domingo_20
Leganés - Atlético	→ Sabado_12
Levante - Getafe	→ Domingo_16
Valladolid - Betis	→ Domingo_12
Barcelona - Eibar	→ Domingo_18
Real Madrid - Villarreal	→ Sabado_20
Mallorca - Valencia	→ Viernes_20
Alavés - Celta	→ Sabado_16
Athletic - Osasuna	→ Lunes_20

Audiencia de la jornada: 6.52 millones de espectadores.

===== Jornada 19 =====

Espanyol - Real Sociedad	→ Sabado_20
Sevilla - Atlético	→ Sabado_18
Granada - Getafe	→ Sabado_16
Leganés - Betis	→ Sabado_12
Levante - Eibar	→ Domingo_12
Valladolid - Villarreal	→ Viernes_20
Barcelona - Valencia	→ Domingo_20
Real Madrid - Celta	→ Domingo_18
Mallorca - Osasuna	→ Lunes_20
Alavés - Athletic	→ Domingo_16

Audiencia de la jornada: 6.86 millones de espectadores.

===== Jornada 20 =====

Atlético - Espanyol	→ Sabado_18
Getafe - Real Sociedad	→ Sabado_20
Betis - Sevilla	→ Domingo_16
Eibar - Granada	→ Lunes_20
Villarreal - Leganés	→ Sabado_12
Valencia - Levante	→ Sabado_16
Celta - Valladolid	→ Domingo_12
Osasuna - Barcelona	→ Domingo_18
Athletic - Real Madrid	→ Domingo_20
Alavés - Mallorca	→ Viernes_20

Audiencia de la jornada: 6.71 millones de espectadores.

===== Jornada 21 =====

Getafe - Espanyol	→ Sabado_18
Betis - Atlético	→ Domingo_16
Eibar - Real Sociedad	→ Domingo_20
Villarreal - Sevilla	→ Sabado_16
Valencia - Granada	→ Sabado_12
Celta - Leganés	→ Domingo_12
Osasuna - Levante	→ Viernes_20
Athletic - Valladolid	→ Lunes_20
Alavés - Barcelona	→ Sabado_20
Mallorca - Real Madrid	→ Domingo_18

Audiencia de la jornada: 6.52 millones de espectadores.

===== Jornada 22 =====

Betis - Espanyol	→ Domingo_18
Eibar - Getafe	→ Sabado_12
Villarreal - Atlético	→ Sabado_18
Valencia - Real Sociedad	→ Domingo_20
Celta - Sevilla	→ Domingo_16
Osasuna - Granada	→ Viernes_20
Athletic - Leganés	→ Domingo_12
Alavés - Levante	→ Sabado_16
Mallorca - Valladolid	→ Lunes_20
Real Madrid - Barcelona	→ Sabado_20

Audiencia de la jornada: 7.22 millones de espectadores.

===== Jornada 23 =====

Eibar - Espanyol	→ Domingo_12
Villarreal - Betis	→ Sabado_18
Valencia - Getafe	→ Domingo_16
Celta - Atlético	→ Sabado_16
Osasuna - Real Sociedad	→ Domingo_20
Athletic - Sevilla	→ Sabado_12
Alavés - Granada	→ Viernes_20
Mallorca - Leganés	→ Lunes_20
Real Madrid - Levante	→ Sabado_20
Barcelona - Valladolid	→ Domingo_18

Audiencia de la jornada: 6.50 millones de espectadores.

===== Jornada 24 =====

Villarreal - Espanyol	→ Sabado_18
Valencia - Eibar	→ Sabado_12
Celta - Betis	→ Domingo_16
Osasuna - Getafe	→ Domingo_12
Athletic - Atlético	→ Sabado_16
Alavés - Real Sociedad	→ Sabado_20
Mallorca - Sevilla	→ Viernes_20
Real Madrid - Granada	→ Domingo_20
Barcelona - Leganés	→ Domingo_18
Valladolid - Levante	→ Lunes_20

Audiencia de la jornada: 6.52 millones de espectadores.

===== Jornada 25 =====

Valencia - Espanyol	→ Domingo_18
Celta - Villarreal	→ Sabado_18
Osasuna - Eibar	→ Viernes_20
Athletic - Betis	→ Domingo_16
Alavés - Getafe	→ Sabado_16
Mallorca - Atlético	→ Sabado_12
Real Madrid - Real Sociedad	→ Sabado_20
Barcelona - Sevilla	→ Domingo_20
Valladolid - Granada	→ Lunes_20
Levante - Leganés	→ Domingo_12

Audiencia de la jornada: 7.22 millones de espectadores.

===== Jornada 26 =====

Celta - Espanyol	→ Sabado_18
Osasuna - Valencia	→ Sabado_12
Athletic - Villarreal	→ Domingo_16
Alavés - Eibar	→ Domingo_12
Mallorca - Betis	→ Viernes_20
Real Madrid - Getafe	→ Sabado_20
Barcelona - Atlético	→ Domingo_20
Valladolid - Real Sociedad	→ Domingo_18
Levante - Sevilla	→ Sabado_16
Leganés - Granada	→ Lunes_20

Audiencia de la jornada: 6.71 millones de espectadores.

===== Jornada 27 =====

Osasuna - Espanyol	→ Sabado_12
Athletic - Celta	→ Sabado_18
Alavés - Valencia	→ Domingo_16
Mallorca - Villarreal	→ Domingo_12
Real Madrid - Eibar	→ Domingo_20
Barcelona - Betis	→ Sabado_20
Valladolid - Getafe	→ Viernes_20
Levante - Atlético	→ Sabado_16
Leganés - Real Sociedad	→ Domingo_18
Granada - Sevilla	→ Lunes_20

Audiencia de la jornada: 6.52 millones de espectadores.

===== Jornada 28 =====

Athletic - Espanyol	→ Sabado_18
Alavés - Osasuna	→ Sabado_16
Mallorca - Celta	→ Sabado_12
Real Madrid - Valencia	→ Sabado_20
Barcelona - Villarreal	→ Domingo_20
Valladolid - Eibar	→ Lunes_20
Levante - Betis	→ Domingo_16
Leganés - Getafe	→ Domingo_12
Granada - Atlético	→ Viernes_20
Sevilla - Real Sociedad	→ Domingo_18

Audiencia de la jornada: 6.86 millones de espectadores.

===== Jornada 29 =====

Alavés - Espanyol	→ Sabado_18
Mallorca - Athletic	→ Sabado_12
Real Madrid - Osasuna	→ Domingo_18
Barcelona - Celta	→ Sabado_20
Valladolid - Valencia	→ Domingo_12
Levante - Villarreal	→ Domingo_16
Leganés - Eibar	→ Lunes_20
Granada - Betis	→ Viernes_20
Sevilla - Getafe	→ Sabado_16
Real Sociedad - Atlético	→ Domingo_20

Audiencia de la jornada: 6.71 millones de espectadores.

===== Jornada 30 =====

Mallorca - Espanyol	→ Sabado_16
Real Madrid - Alavés	→ Sabado_20
Barcelona - Athletic	→ Domingo_20
Valladolid - Osasuna	→ Lunes_20
Levante - Celta	→ Sabado_18
Leganés - Valencia	→ Sabado_12
Granada - Villarreal	→ Domingo_12
Sevilla - Eibar	→ Viernes_20
Real Sociedad - Betis	→ Domingo_18
Atlético - Getafe	→ Domingo_16

Audiencia de la jornada: 6.86 millones de espectadores.

===== Jornada 31 =====

Real Madrid - Espanyol	→ Sabado_20
Barcelona - Mallorca	→ Domingo_18
Valladolid - Alavés	→ Sabado_12
Levante - Athletic	→ Sabado_18
Leganés - Osasuna	→ Lunes_20
Granada - Celta	→ Domingo_12
Sevilla - Valencia	→ Domingo_16
Real Sociedad - Villarreal	→ Domingo_20
Atlético - Eibar	→ Viernes_20
Getafe - Betis	→ Sabado_16

Audiencia de la jornada: 6.71 millones de espectadores.

===== Jornada 32 =====

Barcelona - Espanyol	→ Sabado_20
Valladolid - Real Madrid	→ Domingo_18
Levante - Mallorca	→ Sabado_16
Leganés - Alavés	→ Sabado_12
Granada - Athletic	→ Domingo_12
Sevilla - Osasuna	→ Viernes_20
Real Sociedad - Celta	→ Domingo_20
Atlético - Valencia	→ Sabado_18
Getafe - Villarreal	→ Domingo_16
Betis - Eibar	→ Lunes_20

Audiencia de la jornada: 6.72 millones de espectadores.

===== Jornada 33 =====

Valladolid - Espanyol	→ Sabado_12
Levante - Barcelona	→ Sabado_20
Leganés - Real Madrid	→ Domingo_18
Granada - Mallorca	→ Lunes_20
Sevilla - Alavés	→ Sabado_18
Real Sociedad - Athletic	→ Domingo_20
Atlético - Osasuna	→ Domingo_12
Getafe - Celta	→ Domingo_16
Betis - Valencia	→ Sabado_16
Eibar - Villarreal	→ Viernes_20

Audiencia de la jornada: 6.71 millones de espectadores.

===== Jornada 34 =====

Levante - Espanyol	→ Sabado_18
Leganés - Valladolid	→ Lunes_20
Granada - Barcelona	→ Domingo_20
Sevilla - Real Madrid	→ Sabado_20
Real Sociedad - Mallorca	→ Domingo_18
Atlético - Alavés	→ Domingo_16
Getafe - Athletic	→ Sabado_16
Betis - Osasuna	→ Domingo_12
Eibar - Celta	→ Viernes_20
Villarreal - Valencia	→ Sabado_12

Audiencia de la jornada: 6.50 millones de espectadores.

===== Jornada 35 =====

Leganés - Espanyol	→ Domingo_16
Granada - Levante	→ Sabado_16
Sevilla - Valladolid	→ Sabado_12
Real Sociedad - Barcelona	→ Sabado_20
Atlético - Real Madrid	→ Domingo_20
Getafe - Mallorca	→ Domingo_12
Betis - Alavés	→ Domingo_18
Eibar - Athletic	→ Viernes_20
Villarreal - Osasuna	→ Lunes_20
Valencia - Celta	→ Sabado_18

Audiencia de la jornada: 7.22 millones de espectadores.

===== Jornada 36 =====

Granada - Espanyol	→ Sabado_16
Sevilla - Leganés	→ Sabado_12
Real Sociedad - Levante	→ Sabado_20
Atlético - Valladolid	→ Domingo_12
Getafe - Barcelona	→ Domingo_20
Betis - Real Madrid	→ Domingo_18
Eibar - Mallorca	→ Lunes_20
Villarreal - Alavés	→ Sabado_18
Valencia - Athletic	→ Domingo_16
Celta - Osasuna	→ Viernes_20

Audiencia de la jornada: 6.86 millones de espectadores.

===== Jornada 37 =====

Sevilla - Espanyol	→ Sabado_18
Real Sociedad - Granada	→ Domingo_20
Atlético - Leganés	→ Sabado_12
Getafe - Levante	→ Domingo_16
Betis - Valladolid	→ Domingo_12
Eibar - Barcelona	→ Domingo_18
Villarreal - Real Madrid	→ Sabado_20
Valencia - Mallorca	→ Viernes_20
Celta - Alavés	→ Sabado_16
Osasuna - Athletic	→ Lunes_20

Audiencia de la jornada: 6.52 millones de espectadores.

===== Jornada 38 =====

Real Sociedad - Espanyol	→ Sabado_20
Atlético - Sevilla	→ Sabado_18
Getafe - Granada	→ Sabado_16
Betis - Leganés	→ Sabado_12
Eibar - Levante	→ Domingo_12
Villarreal - Valladolid	→ Viernes_20
Valencia - Barcelona	→ Domingo_20
Celta - Real Madrid	→ Domingo_18
Osasuna - Mallorca	→ Lunes_20
Athletic - Alavés	→ Domingo_16

Audiencia de la jornada: 6.86 millones de espectadores.

=====

Audiencia total del calendario: 256.96 millones de espectadores

=====

Enumera las referencias que has utilizado(si ha sido necesario) para llevar a cabo el trabajo

1. Easton, K., Nemhauser, G., & Trick, M. (2001). The Traveling Tournament Problem Description and Benchmarks (CP 2001)
2. FLM System. (s. f.). Round robin scheduling. <https://www.flmsystem.com/blog/round-robin-scheduling>

2.5.3 Declaración de uso de Inteligencia Artificial (AID)

Con el fin de garantizar la transparencia y la integridad académica, se declara el uso de herramientas de Inteligencia Artificial como apoyo durante la elaboración de este trabajo. La responsabilidad final sobre el contenido, las decisiones adoptadas y las conclusiones presentadas corresponde exclusivamente a los autores.

1. Herramienta utilizada

- ChatGPT (OpenAI, GPT-5.5)
- Gemini

2. Propósito del uso

La herramienta se utilizó como apoyo para:

- Mejora de los algoritmos utilizados.
- Resolver dudas conceptuales relacionadas con los algoritmos voraces y la planificación *Round Robin*.
- Mejorar la redacción y corrección ortográfica del documento.
- Obtener sugerencias sobre la estructura y presentación del trabajo.
- Ayudar en la búsqueda y el formateo de referencias bibliográficas en formato APA.

3. Prompts (instrucciones) utilizados

Algunos ejemplos de las consultas realizadas fueron:

- Como puedo mejorar el siguiente algoritmo para un algoritmo de optimización.
- “Explica el funcionamiento de un algoritmo voraz aplicado a la planificación de torneos Round Robin.”
- “Corrige la redacción y las faltas ortográficas de este apartado.”
- “Revisa la calidad académica del notebook y sugiere posibles mejoras.”

Respuesta

Describe brevemente las líneas de como crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

Respuesta

En este seminario se asume un escenario determinista basado en 10 partidos estáticos. Sin embargo, la organización real del calendario de La Liga es un problema de optimización altamente complejo (catalogado como NP-Hard). Para avanzar en el estudio de este problema y acercarlo a un entorno de producción real, se proponen las siguientes líneas de investigación y variaciones:

1. En la práctica, la asignación de horarios no depende únicamente de la audiencia televisiva, sino de la disponibilidad de recursos logísticos. Se deben modelar restricciones relacionadas con la seguridad ciudadana. Por ejemplo, hay que evitar que dos equipos de la misma ciudad (ej. Real Madrid y Atlético de Madrid) jueguen como locales el mismo día, ya que los recursos policiales y de movilidad urbana no pueden cubrir ambos eventos simultáneamente.
2. El algoritmo actual no tiene en cuenta el estado previo de los equipos. Una variación fundamental consistiría en añadir una variable temporal que conecte las jornadas entre sí, incorporando las competiciones europeas o la Copa del Rey. Esto obligaría al algoritmo a respetar ventanas de descanso obligatorias (mínimo de 72 horas). Si un equipo juega el jueves en Europa, matemáticamente su espacio de soluciones válido para La Liga se reduciría, impidiendo

que se le asigne el horario de máxima audiencia del sábado, y forzando al modelo a buscar un subóptimo global.

3. Si se aumenta el tamaño del problema para incluir simultáneamente la primera y la segunda división, el volumen pasaría a ser de más de 21 partidos por fin de semana. En este volumen, el solapamiento de partidos dejaría de ser evitable y la penalización por coincidencia se activaría obligatoriamente.

Para soportar este incremento drástico en el tamaño del espacio de búsqueda, el algoritmo voraz actual perdería su eficacia. El siguiente paso en el estudio requeriría implementar otro tipo de arquitecturas para abordar la explosión combinatoria. En concreto, se proponen métodos de búsqueda local estocástica, es decir, algoritmos que exploran el espacio de soluciones aplicando modificaciones mediante un factor de azar matemático muy controlado.

Dentro de esta familia, se destacan los algoritmos genéticos. En lugar de construir un único calendario paso a paso, este sistema generaría una “población” inicial de múltiples calendarios completos y simularía un proceso evolutivo. Seleccionando los calendarios que devuelvan mayor audiencia, el algoritmo combinaría sus estructuras e introduciría pequeños cambios aleatorios en los horarios de partidos concretos. Al iterar este ciclo durante miles de generaciones, el sistema descartaría las asignaciones ineficientes y convergería de forma orgánica hacia el calendario global óptimo.